Phishing Detection Using Machine Learning Techniques

Critical Reproduction and Evaluation Study

Student: Saleh

Course: Data Science in Cyber

Source: Shahrivari, Darabi & Izadi (2020) — arXiv:2009.11116

Original repository: https://github.com/fafal-abnir/phishing_detection

1 .SUMMARY OF THE SOURCE

The paper addresses binary classification of phishing versus legitimate websites. Given URL and website-level features, the goal is to predict whether a site is a phishing attempt designed to steal credentials or other sensitive information.

Phishing causes large financial and reputational damage. The authors cite FBI IC3 data ($2.7B losses in 2018) and APWG trends showing sustained growth in phishing sites. Automated detection scales better than user training alone and can block malicious pages before users interact with them.

The proposed solution is to compare twelve supervised machine learning classifiers on a standard phishing feature dataset: Logistic Regression, Decision Tree, Random Forest, AdaBoost, KNN, SVM (four kernels), Neural Network, Gradient Boosting, and XGBoost. The best model is selected by accuracy, precision, recall, F1, and runtime.

The dataset is the UCI Phishing Websites dataset (Mohammad, Thabtah & McCluskey, 2015), with 11,055 instances, 30 pre-extracted features per URL/site, labeled -1 (phishing) or 1 (legitimate). In our copy, approximately 56% of samples are legitimate and 44% are phishing.

The methodology uses 10-fold cross-validation on the full dataset after global feature standardization. Default hyperparameters are used for all models, although the paper mentions tuning. Primary metrics are accuracy, precision, recall, F1, and train/test time. XGBoost is reported as the best model, with about 98.3% accuracy in Table III.

2 .CRITICAL EVALUATION

The main claims made by the authors are that machine learning can effectively detect phishing websites from URL and structure features; that ensemble and boosting methods (Random Forest, XGBoost) outperform simpler classifiers; that XGBoost achieves the highest accuracy and a strong F1 score on this dataset; that feature selection and hyperparameter tuning were applied to obtain the best results; and that the comparative study is reproducible via the published GitHub repository.

These claims are partially supported. Our reproduction of All_Classifair.ipynb yielded similar rankings: ensemble methods beat logistic regression and SVM-sigmoid, and Random Forest and XGBoost reached about 97% accuracy, close to Table III. The general claim that boosting and ensemble methods work well is therefore supported. However, the absolute XGBoost gap (paper 98.3% versus our 97.2% in reproduction and 97.2% in our improved pipeline) suggests that results are sensitive to environment and preprocessing. The claim of extensive hyperparameter tuning is not supported by the code, which uses default estimators.

The evaluation methodology is only partially appropriate. Ten-fold cross-validation on a fixed tabular dataset is a reasonable baseline, although stratification is not explicit in the authors' code (we added StratifiedKFold in our work). Important weaknesses include global scaling before cross-validation, which leaks test-fold statistics into training; the absence of a held-out temporal test set, meaning features may be outdated relative to modern phishing; an overemphasis on accuracy rather than recall on phishing and the cost of false negatives; no discussion of class imbalance (about 44% phishing) or class_weight handling; and the fact that precision and recall in sklearn default to

pos_label=1 (legitimate), while phishing detection usually focuses on catching label -1.

Additional weaknesses and limitations include that roughly half of the paper is background material with limited experimental depth; features are pre-engineered by a third party and the authors do not extract or justify them; there is no error analysis, confusion matrix discussion, or security interpretation of mistakes; Table II is mislabeled as classification results although it shows feature statistics; the neural network section refers to "30 hidden layers," which likely means 30 neurons; the repository is incomplete with missing dependencies, README typos, and no random seeds; the dataset has no timestamps so temporal validation is impossible; and high feature redundancy (12 pairs with Spearman |r| >= 0.7) is not addressed.

The conclusion that ensemble methods are strong baselines on this dataset is justified by both the paper and our reproduction. The implication that XGBoost is clearly superior to all alternatives is only weakly justified, since XGBoost is only marginally ahead of Random Forest and the neural network in the paper, and our runs show Random Forest and XGBoost nearly tied. Claims about rigorous tuning and feature selection are not justified by the published code. Overall, the work is a useful benchmark comparison but not a rigorous experimental study.

3 .FEATURE ENGINEERING ANALYSIS

Feature engineering was performed only minimally in the original paper and repository. Features are pre-extracted by the UCI dataset authors as 30 URL/website attributes encoded as -1, 0, or 1. The authors apply only a global StandardScaler before cross-validation. No encoding, feature creation, or feature selection is documented.

The 30 features come from the UCI Phishing Websites dataset and include SSLfinal_State, URL length, Prefix_Suffix, domain age, and Page_Rank. The target variable is Result, where -1 means phishing and 1 means legitimate.

For transformations, the authors use sklearn.preprocessing.scale() on the full feature matrix before cross-validation. In our pipeline, we placed StandardScaler inside a sklearn Pipeline so scaling is fit on each training fold only, which avoids data leakage. No extra encoding was needed because the features are already numeric. We did not drop correlated features in this baseline, but EDA found 12 pairs with Spearman |r| >= 0.7, including Favicon and popUpWidnow (r = 0.94).

Redundancy was detected through Spearman correlation and high-correlation pairs. Redundant URL-structure signals could be pruned (for example, keeping one of Favicon or popUpWidnow) or handled with regularization, which tree models partly do implicitly. The authors do not discuss redundancy.

The feature engineering is partially meaningful. The underlying UCI features capture real phishing indicators such as suspicious SSL, URL tricks, and domain metadata. The authors' only transformation, scaling, helps distance-based models such as SVM, KNN, and logistic regression, but applying it globally is methodologically weak. Tree-based models are less sensitive to scaling.

Additional features that could improve performance include fresh lexical URL tokens, certificate transparency logs, page content embeddings, reputation feeds, and temporal features such as domain age in days and time since certificate issue. The paper itself suggests adding more features in future work.

4 .REPRODUCIBILITY ANALYSIS

The code can be executed successfully only after fixes. After installing missing packages (xgboost, lightgbm, catboost) and remapping labels from {-1, 1} to {0, 1} for XGBoost, the notebook runs end-to-end. The original repository does not document these steps.

Required files and dependencies are mostly available. The dataset dataset.csv is included in the authors' repository (11,055 rows, 30 features plus label). All_Classifair.ipynb and dataset_description.ipynb are provided. However, requirment.txt lists only numpy, scikit-learn, beautifulsoup4, whois, google,

seaborn, and pandas, while omitting xgboost, lightgbm, and catboost, which are imported in the main notebook. The README also references "requirments.txt" (misspelled) instead of "requirment.txt."

Hidden preprocessing steps include applying global StandardScaler to all features before cross-validation through preprocessing.scale on the full matrix X, which is not emphasized in the paper and can leak fold statistics into training. The dataframe is shuffled with sklearn.utils.shuffle() without a random_state, so runs are not exactly reproducible across sessions. Label encoding uses phishing = -1 and legitimate = 1, but newer XGBoost versions require {0, 1} and fail without remapping.

Overall reproducibility is moderate. The same dataset and general pipeline reproduce results that are close to the paper (within about 1–2% accuracy for most models), but exact numbers differ because of library versions, unseeded shuffling, and environment fixes not included in the original repo. A reader cannot fully reproduce the paper from the README alone without troubleshooting.

5 .EXPERIMENTAL RESULTS

We performed two sets of experiments. First, we re-ran All_Classifair.ipynb from the authors' GitHub repository using 10-fold cross-validation and the same default classifiers. Second, we built our own pipeline in project.ipynb using stratified 10-fold cross-validation, random_state=42, StandardScaler inside a Pipeline, three models (Logistic Regression, Random Forest, and XGBoost), metrics with phishing as the positive class, and confusion matrix and error analysis.

For the reproduction run, modifications included installing xgboost, lightgbm, and catboost, and remapping labels for XGBoost. In our study, we added per-fold scaling, fixed seeds, StratifiedKFold, MCC, phishing-focused precision and recall, and out-of-fold predictions for error analysis.

The authors' notebook trains more than twelve classifiers, including trees, boosting methods, SVM kernels, and an MLP. Our notebook trains Logistic Regression, Random Forest, and XGBoost.

The authors report accuracy, precision, recall, F1, and train/test time. We report accuracy, precision, recall, F1, and MCC with phishing as the positive class, plus confusion-matrix-based false positive and false negative counts.

Reproduction results compared to the paper (accuracy) were as follows: Decision Tree 0.965 (paper 0.966), Random Forest 0.973 (paper 0.973), XGBoost 0.972 (paper 0.983), Logistic Regression 0.927 (paper 0.927), Neural Network 0.970 (paper 0.970), and SVM RBF 0.952 (paper 0.952).

Our pipeline results (10-fold cross-validation) were: Logistic Regression — accuracy 0.928, precision 0.929, recall 0.907, F1 0.918, MCC 0.854; Random Forest — accuracy 0.971, precision 0.976, recall 0.959, F1 0.967, MCC 0.942; XGBoost — accuracy 0.972, precision 0.975, recall 0.963, F1 0.969, MCC 0.944.

Error analysis on XGBoost using out-of-fold predictions found 182 false negatives (missed phishing) and 123 false positives (legitimate sites flagged). Missed phishing sites differed most on SSLfinal_State, URL_of_Anchor, and web_traffic compared with correctly detected phishing, suggesting that borderline SSL and URL cases drive many errors.

These results support the authors' ranking, with ensemble methods clearly outperforming logistic regression, but they also show that high accuracy still leaves hundreds of missed phishing URLs in a dataset of 11,000 sites.

6 .CONCLUSIONS

The Shahrivari et al. study is a reproducible benchmark on the UCI Phishing Websites dataset rather than a novel detection method. Ensemble models such as Random Forest and XGBoost achieve about 97% accuracy, and our improved

pipeline confirms this with proper per-fold scaling and phishing-focused metrics. Reproducibility is moderate: the code runs after fixes, but dependencies, seeds, and label encoding issues are not documented. Even strong models miss about 3.7% of phishing sites (182 false negatives), which is meaningful at scale.

We learned that reproducing a paper requires checking the repository, not only the PDF, because hidden preprocessing and missing packages matter. Scaling must be applied inside cross-validation to avoid leakage. For cybersecurity classification, metric choice (recall on phishing, MCC, and FP/FN trade-offs) is as important as accuracy. EDA also revealed substantial feature redundancy that the authors never discuss.

Strengths of the proposed solution include a simple pipeline, fast training, strong baselines on a public dataset, comparison of multiple models, and open data and code. Weaknesses include an outdated feature set, minimal feature engineering, weak experimental rigor (no stratification, global scaling, no error analysis in the original work), overstated claims about tuning, and no discussion of deployment or concept drift.

Suggestions for future improvements include adding fresh features (URL tokens, certificates, reputation APIs, page content), using temporal or geographically held-out test sets, reporting confusion matrices, MCC, and ROC-AUC with phishing as the explicit positive class, addressing redundancy through feature selection or PCA, and comparing against modern phishing detectors while discussing adversarial URL evasion.

7 .EXECUTIVE SUMMARY

This project critically evaluated and reproduced "Phishing Detection Using Machine Learning Techniques" (Shahrivari et al., 2020), which compares twelve machine learning classifiers on the UCI Phishing Websites dataset of about 11,000 URLs and 30 hand-crafted features.

We reproduced the authors' Jupyter notebook and built an improved analysis pipeline. Reproduction succeeded after installing undeclared dependencies and fixing XGBoost label encoding. Results broadly matched Table III, for example Random Forest at about 97.3% accuracy and XGBoost at about 97.2% compared with 98.3% in the paper. Our pipeline added stratified cross-validation, per-fold scaling, fixed random seeds, and security-oriented metrics.

The paper's main claim that ensemble and boosting methods outperform simpler models is supported. However, critical weaknesses include global scaling before cross-validation (data leakage risk), no reproducibility seeds, overstated hyperparameter tuning, missing error analysis, and no discussion of false negatives in a security context. Feature engineering is minimal; the value lies in the UCI features, not in the authors' transformations.

XGBoost achieved the best overall performance in our study (97.2% accuracy, 96.3% phishing recall, MCC 0.944) but still missed 182 phishing sites. We recommend this benchmark for teaching and baseline comparison, not as a production-ready detector without substantial feature updates and rigorous security-focused evaluation.

8 .SUMMING IT UP

The problem addressed is automated detection of phishing websites using supervised learning on URL and website features.

The selected article is Shahrivari, Darabi & Izadi (2020), arXiv:2009.11116, with code at github.com/fafal-abnir/phishing_detection.

The dataset used is the UCI Phishing Websites dataset with 11,055 samples, 30 features, and a binary label (-1 for phishing, 1 for legitimate).

Our methodology reproduced the authors' 10-fold cross-validation benchmark and extended it with stratified cross-validation, Pipeline scaling, three models, MCC, a confusion matrix, and false negative/false positive error analysis.

The main findings of the reproduction study are that results closely match the paper's model ranking, with XGBoost and Random Forest leading, but reproducibility requires undeclared fixes and the original methodology has leakage and reporting gaps.

The authors' claims are supported regarding model ranking and the general effectiveness of ensemble methods, but not regarding full reproducibility from the README, rigorous tuning, or feature selection.

The most important insights are that high accuracy can still leave hundreds of undetected phishing URLs, that security metrics and error analysis matter more than accuracy alone, and that feature redundancy and stale features limit real-world applicability.

For similar problems, we recommend using this work as a baseline benchmark on the UCI dataset, but extending features, fixing cross-validation methodology, and evaluating with phishing recall and operational FP/FN costs before trusting deployment.

In conclusion, the article provides a useful and mostly reproducible comparison study whose main conclusions about ensemble performance hold, but whose experimental rigor and reproducibility documentation fall short of best practice. Our analysis confirms the broad results while identifying methodological improvements essential for credible cybersecurity machine learning.